Access-Mode

ID_3 – Operator / Inspection & Freeze Discipline

Companion to Version E.2 — *Artifact Lifecycle Companion to the Access-Mode Method*

APOD-ID3-OR-001 • Informative • Non-Normative

Document Control Information

Document Identifier: APOD-ID3-OR-001
Title: Access-Mode ID_3 — Operator / Inspection & Freeze Discipline
Edition: Companion Handbook — Version ID_3
Date of Issue: 27 February 2026
Status: Informative Companion — Non-Normative
Author: Anthony Siljeg
Affiliation: Alnotrea Labs / Policy of Developments — Access PoD *Initiative*

Abstract

Access-Mode ID_2 is a structured operational reasoning posture for capturing governance signals during live session runs. ID_2 artifacts intentionally preserve exploratory reasoning, uncertainty articulation, drift indicators, and escalation triggers. They are non-inspection records and do not assert adequacy, compliance, approval, or regulatory standing.

In practice, baseline and variant ID_2 sessions may diverge in metadata completeness, identity fields, timestamps, or structural consistency, particularly under time pressure or iterative refinement. Promoting such records without inspection discipline risks structural ambiguity, provenance gaps, and non-deterministic packaging outcomes.

Access-Mode ID_3 is the downstream inspection and freeze discipline that prepares operational records for inspection, archival integrity, and standards-facing review, without modifying underlying reasoning content.

ID_3 performs:

• Structural alignment checks against a defined session completeness rule set
• Metadata-only normalization of identity fields, timestamps, and array structure
• Deterministic canonical promotion of selected artifacts
• Freeze stamping under ID_3 semantics
• Generation of a checksum-based integrity anchor, for example hashes.txt

ID_3 does not:

- Rewrite reasoning content
- Alter operational captures
- Assert compliance, adequacy, certification, or approval
- Convert inspection findings into governance decisions

Within ID_3, the term “final” refers exclusively to final packaging under ID_3 freeze semantics for hashing, archival verification, and reproducibility. It does not imply compliance, endorsement, regulatory sufficiency, or audit completion.

ID_3 operates under a formalized state model that distinguishes operational session states (ID_2), inspection states (ID_3), and canonical packaging states. This separation improves teachability, reproducibility, and regulatory readability.

Once inspection alignment is complete, ID_3 issues canonical JSON artifacts and a deterministic hash manifest, establishing a verifiable inspection-ready surface suitable for workspace sealing, archive verification, and standards preparation.

Practical Execution Note (Informative)

Access-Mode ID_3 (Inspect → Freeze → Hash) is designed to be operationally lightweight once session material is prepared. A typical inspection cycle may be completed in approximately 15 minutes, excluding time required for human review of session content.

This Companion document explains the full structural discipline behind ID_3, including:

• inspection boundaries
• rule-governed metadata correction
• canonical promotion semantics
• deterministic freeze and integrity anchoring

For structural enforcement semantics, refer to the standalone Rule Engine release v1.1:
https://github.com/Access-PoD/access-pod-artifacts/releases/tag/APOD-ID3-RULE-001-v1.1

For practical execution, including scripts and toolkit examples, refer to the official ID_3 operator release:
https://github.com/Access-PoD/access-pod-artifacts/releases/tag/ID_3-operator

The GitHub release contains:

• the full ID_3 design implementation
• inspection workflow scripts
• and the ID_3 Toolkit Script Examples Companion

This document provides architectural and methodological clarity.
The GitHub release provides execution mechanics.

Relationship to Version E and Version E.2

This Companion document is subordinate to and consistent with:

- **APOD-TR-006 — A WEB of Sharing Trust–Responsibility Toward a Trustless Future, Vol. 2 — Version E (Access-Mode Method)**
- **APOD-TR-006-E.2 — Artifact Lifecycle Companion to the Access-Mode Method**

Version E defines the Access-Mode method, including:

- reasoning posture discipline (ID_1, ID_2, ID_3),
- cross-mode constraints,
- validation and misalignment testing principles,
- explicit non-normative boundaries.

Version E.2 clarifies the lifecycle of Working-PoD artifacts, distinguishing:

- reasoning posture from artifact state,
- draft and populated states from frozen inspection state,
- inspection readiness from authority or adequacy.

This ID_3 Companion does not modify, reinterpret, or extend either document.

Instead:

- Version E governs how reasoning occurs.
- Version E.2 governs what artifact state represents.
- This Companion governs how ID_3 inspection posture is operationalized as structural packaging and freeze discipline.

Where ambiguity arises:

- Version E governs reasoning posture.
- Version E.2 governs artifact lifecycle semantics.
- This Companion governs inspection implementation mechanics.

This document remains informative and non-normative.
It does not define requirements, conformance criteria, compliance status, or certification pathways.

Public Releases (Reference)

The Access-Mode method and associated module sets are published in the Access PoD public repository.

Primary method release (Version E and E.2):
https://github.com/Access-PoD/access-pod-artifacts/releases/tag/v1.0-access-mode

Single-mode releases:

- **Access-Mode ID_1 — Core (Learning)**
  https://github.com/Access-PoD/access-pod-artifacts/releases/tag/ID_1-core
- **Access-Mode ID_2 — Operational / Runtime Reasoning**
  https://github.com/Access-PoD/access-pod-artifacts/releases/tag/ID_2-Access

These releases provide module sets, templates, and execution artifacts referenced by this Companion.

Contents

Abstract 1

Practical Execution Note (Informative) 3

Relationship to Version E and Version E.2 3

Public Releases (Reference) 4

1. Purpose and Scope 9

1.1 Purpose 9

1.2 Scope 9

1.3 What ID_3 Is 10

1.4 What ID_3 Is Not 10

1.5 Why Inspection Packaging Requires Discipline 11

2. ID_2 vs ID_3 Boundary 12

2.1 Distinct Cognitive Functions 12

2.2 Non-Inspection Capture (ID_2) 12

2.3 Inspection Packaging (ID_3) 13

2.4 Prohibited Transformations Under ID_3 13

2.5 The Structural Correction Boundary 14

2.6 Why This Boundary Matters 14

3. Formal State Model 16

3.1 Why State Formalization Is Required 16

3.2 ID_2 Operational States 16

3.3 ID_3 Inspection States 17

3.4 Canonical Packaging State 17

3.4A Integrity Anchor and Canonical Freeze Surface 18

3.4B Relationship to 10_working and 20_corrected Surfaces 19

10_working - FOLDER 19

20_corrected - FOLDER 19

3.4C Canonical Promotion and Hash/Freeze Step 19

3.5 State Diagram (Illustrative Model) 20

3.6 State Stability and Reuse Discipline 21
4. ID_3 Rule Engine Overview 22
4.1 Purpose of the Rule Engine 22
4.2 Implementation Form 22
4.3 Structural Constraint Categories 23
A. Required Structural Constraints 23
B. Optional Quality Signals 23
C. Non-Fabrication Constraints 24
4.4 Diagnostic vs Evaluative Behavior 24
4.5 Relationship to ID_2 Artifacts 25
4.6 Correction Workflow Alignment 25
4.7 Versioning and Extensibility 26
4.8 Why Formalization Matters 27
5. Verification Workflow 28
Operational Sequence Under Rule Governance 28
5.1 Purpose of the Verification Workflow 28
5.2 Workflow Overview 28
5.3 Step 1 — Verify (Intake Surface) 28
5.4 Step 2 — Correct (Normalization Surface) 29
5.5 Step 3 — Re-Verify 29
5.6 Step 4 — Canonical Promotion 30
5.7 Step 5 — Freeze Declaration 30
5.8 Step 6 — Hash Generation (Integrity Anchor) 31
5.9 Outbox Preservation and Provenance 31
5.10 Workflow Discipline Summary 32
6. Canonical Promotion Semantics 33
6.1 Purpose of Canonical Promotion 33
6.2 Inspection Support Artifacts (Pre-Freeze Records) 33
6.3 Canonical Artifact Generation 34

6.4 Inspection Status Fields 35
6.5 Freeze Declaration 35
6.6 Integrity Anchoring 35
6.7 Separation of Workflow Records and Canonical Surface 36
7. Deterministic Integrity and Archive Model 37
7.1 Purpose of the Integrity Model 37
7.2 Deterministic Hash Surface 37
7.3 Verification Procedure 38
7.4 Outbox Archive Discipline 38
7.5 Reproducibility Expectations 39
7.6 Integrity vs Authority 39
7.7 Interaction with Future Sessions 40
7.8 Governance Value of Deterministic Archiving 40
8. Provenance and Reproducibility 41
8.1 Purpose of Provenance Discipline 41
8.2 10_working as Evidentiary Intake Surface 41
8.3 20_corrected as Normalization Boundary 42
8.4 Canonical Surface as Freeze Record 42
8.5 Lineage Across Sessions 43
8.6 Reuse as New ID_2 Baseline 43
8.7 Provenance vs Authority 44
9. Troubleshooting and Structural Edge Cases 45
9.1 Purpose of Troubleshooting Discipline 45
9.2 Array Scalarization Issues 45
Problem 45
Permitted Correction 45
9.3 Timestamp Coercion Issues 46
Problem 46
Permitted Correction 46

9.4 Placeholder Identity Handling 47
Problem 47
Permitted Correction 47
9.5 Optional Field Emptiness 47
9.6 Schema Drift Detection 48
9.7 Completeness Scoring Misinterpretation 48
9.8 When to Revert to ID_2 49
9.9 Freeze Invalidation Scenarios 49
9.10 Structural Troubleshooting Summary 50
10. Operational Discipline Statement 51
10.1 ID_3 as Inspection Packaging Discipline 51
10.2 Explicit Non-Claims 51
10.3 Compatibility with Standards Preparation 52
10.4 Human Authority and Review Requirement 52
10.5 Final Positioning 53
Executive Closing 54

1. Purpose and Scope

1.1 Purpose

The purpose of Access-Mode ID_3 is to establish a disciplined inspection and freeze framework for governance artifacts produced under ID_2 operational reasoning.

ID_3 exists to:

- Separate exploratory reasoning from inspection packaging
- Eliminate structural ambiguity prior to archival or release
- Preserve provenance without altering reasoning content
- Enable deterministic, reproducible artifact promotion
- Support regulator-facing and standards-facing preparation

ID_3 formalizes packaging readiness without converting governance drafts into compliance claims.

It introduces:

- Explicit inspection states
- Defined correction boundaries
- Controlled metadata normalization
- Canonical promotion semantics
- Integrity anchoring via deterministic hashing

ID_3 transforms “working artifacts” into “inspection-ready artifacts” without transforming the substance of governance reasoning.

1.2 Scope

ID_3 applies to:

- ID_2 session records selected for inspection packaging
- Baseline and variant operational artifacts
- Metadata normalization and structural correction
- Canonical promotion and freeze stamping
- Deterministic manifest and hash generation

ID_3 does not apply to:

- Operational reasoning generation

- Governance decision-making
- Legal interpretation
- Compliance determination
- Certification or approval workflows
- Runtime system behavior

ID_3 governs packaging discipline only.

1.3 What ID_3 Is

ID_3 is:

- An inspection-only discipline
- A structural alignment mechanism
- A metadata normalization framework
- A freeze protocol
- A deterministic packaging gate
- A reproducibility control

It is designed for:

- Standards exploration
- Regulator-facing artifact preparation
- Governance audit readiness
- Workspace sealing and archive integrity

1.4 What ID_3 Is Not

ID_3 is not:

- A compliance engine
- A certification framework
- A regulatory submission tool
- A legal interpretive authority
- A governance decision-maker
- A runtime control system

Inspection findings under ID_3 remain draft observations requiring human judgment.

1.5 Why Inspection Packaging Requires Discipline

Without formal inspection discipline:

- Operational drafts may be promoted with structural inconsistency
- Identity fields may drift across iterations
- Timestamp formats may be non-deterministic
- Arrays may collapse into scalars
- Freeze states may be ambiguous
- Packaging signals may unintentionally imply authority

These risks are structural, not conceptual.

ID_3 exists to control structural risk without altering reasoning substance.

Inspection packaging is treated as a governance control, not a clerical step.

---

2. ID_2 vs ID_3 Boundary

2.1 Distinct Cognitive Functions

ID_2 and ID_3 are distinct reasoning postures as defined in APOD-TR-006 — Version E.

They are not progressive stages.
They answer different governance questions.

- **ID_2 asks:**
  What happens under use, stress, uncertainty, or change?
- **ID_3 asks:**
  How will this be read, structured, interpreted, or inspected?

ID_2 produces exploratory operational artifacts.
ID_3 prepares artifacts for inspection without altering their reasoning content.

The boundary is structural, not hierarchical.

2.2 Non-Inspection Capture (ID_2)

Under ID_2:

- Reasoning is exploratory and uncertainty-aware.
- Scenarios may be provisional.
- Drift, fragility, and ambiguity are surfaced.
- Completeness does not imply adequacy.
- Artifacts remain non-inspectable by default.

ID_2 artifacts may include:

- Hypothetical stress cases
- Conditional reasoning
- Unresolved questions
- Partial metadata
- Iterative identity fields

These characteristics are expected and appropriate under ID_2 posture.

ID_2 does not attempt:

- Structural normalization for archival use

- Deterministic packaging
- Freeze stamping
- Integrity anchoring
- Reviewer-facing finalization

ID_2 artifacts are working governance reasoning records.

2.3 Inspection Packaging (ID_3)

ID_3 applies only after operational reasoning is complete enough to be packaged.

ID_3:

- Does not evaluate operational adequacy
- Does not reinterpret runtime reasoning
- Does not resolve operational uncertainty
- Does not validate governance sufficiency

Instead, ID_3:

- Aligns structure
- Normalizes metadata
- Enforces completeness constraints
- Freezes inspection-ready artifacts
- Generates integrity anchors

Inspection posture improves interpretability, not authority.

2.4 Prohibited Transformations Under ID_3

The following transformations are explicitly prohibited under ID_3:

- Rewriting operational reasoning conclusions
- Downgrading uncertainty to confidence
- Introducing new operational analysis
- Inferring compliance or adequacy
- Filling missing reasoning gaps with synthetic content
- Treating inspection clarity as validation

If such activity is required, the process must revert to ID_2.

2.5 The Structural Correction Boundary

ID_3 permits correction only within defined boundaries:

Permitted:

- Metadata normalization
- Array coercion
- Timestamp formatting
- Identity placeholder replacement (deterministic only)
- Canonical JSON ordering

Not permitted:

- Fabrication of missing reasoning
- Deriving sign-off timestamps from filesystem metadata
- Introducing unrecorded decisions
- Suppressing recorded gaps

This boundary aligns with the structural constraints defined in `session_completeness.rule.json`.

The rule engine is diagnostic and structural.
It is not a reasoning evaluator.

2.6 Why This Boundary Matters

Without an explicit ID_2 / ID_3 boundary:

- Operational drafts may be mistaken for inspection conclusions.
- Structural clarity may be misread as adequacy.
- Freeze state may be misinterpreted as approval.

Separating operational reasoning from inspection packaging:

- Preserves uncertainty in its original context,
- Makes structural correction auditable,
- Prevents authority escalation through formatting.

ID_3 governs structural discipline only.

3. Formal State Model

3.1 Why State Formalization Is Required

In practice, governance artifacts evolve across reasoning sessions.
Without explicit state signaling, structural transitions can become ambiguous.

Operational drafts may be mistaken for inspection-ready artifacts.
Inspection-ready artifacts may be mistaken for validated outcomes.
Packaging signals may be interpreted as approval signals.

To prevent these risks, ID_3 introduces an explicit state model that distinguishes:

- operational artifact states (ID_2),
- inspection states (ID_3), and
- canonical packaging state.

The purpose of this model is not to impose progression or maturity.
It exists to make artifact condition explicit, teachable, and reviewable.

State clarity improves:

- interpretability,
- reproducibility,
- regulatory readability,
- and boundary enforcement between reasoning and packaging.

3.2 ID_2 Operational States

Artifacts produced under ID_2 operational reasoning may exist in one of three descriptive states:

- **draft** — exploratory, evolving, not structurally aligned
- **active** — currently used for operational reasoning or scenario development
- **archived** — retained for reference but not in active iteration

These states reflect operational workflow, not authority or adequacy.

Artifacts in ID_2 states:

- may contain uncertainty,
- may contain partial metadata,

- may contain iterative identity adjustments,
- are non-inspectable by default.

Transition between these states is contextual and human-controlled.

None of these states imply inspection readiness.

3.3 ID_3 Inspection States

When artifacts are selected for inspection packaging, they transition into ID_3 states.

ID_3 introduces two structural states:

- **corrected** — metadata normalized, structural alignment verified
- **frozen_under_ID_3** — inspection-ready, hash-anchored, stable as issued

The corrected state reflects structural alignment only.
Reasoning content remains unchanged.

The frozen_under_ID_3 state indicates:

- explicit freeze declaration,
- canonical promotion,
- integrity anchoring,
- suspension of modification without version increment.

Freezing is deliberate.
It does not emerge automatically from completeness.

3.4 Canonical Packaging State

After freeze stamping and integrity anchoring, artifacts may enter a canonical packaging state:

- **final (packaging semantics only)**

In ID_3 context, “final” refers strictly to:

- finalized packaging under ID_3 freeze discipline,
- deterministic artifact structure,
- reproducible hash manifest.

It does not mean:

- compliance,
- approval,
- adequacy,
- certification,
- readiness for deployment.

This distinction is critical.

3.4A Integrity Anchor and Canonical Freeze Surface

When an ID_3 inspection cycle reaches freeze state, a deterministic canonical surface is produced.

The canonical freeze surface contains:

- `session_baseline.canonical.json`
- `session_variant.canonical.json`
- `module_03_baseline.canonical.json`
- `module_03_variant.canonical.json`
- `hashes.txt`

These artifacts represent the structurally aligned, inspection-ready versions of the selected ID_2 records.

The file `hashes.txt` serves as the deterministic integrity anchor for the frozen pack.

It provides:

- ordered hashing of canonical artifacts,
- reproducible verification surface,
- tamper-evident inspection reference,
- archive integrity validation.

The integrity anchor applies to the canonical surface only.
It does not hash exploratory ID_2 drafts.

Freeze integrity is therefore anchored at the canonical layer, not at the working layer.

3.4B Relationship to 10_working and 20_corrected Surfaces

The ID_3 pack preserves provenance across two internal surfaces:

10_working - FOLDER

- Contains intake copies of ID_2 artifacts.
- Preserves original `session.json` files.
- Preserves original `module_03` artifacts.
- Remains immutable once the inspection cycle begins.
- Serves as evidentiary intake surface.

No structural normalization occurs in this folder.

20_corrected - FOLDER

- Contains metadata-normalized versions of the same artifacts.
- `session.json` files are structurally aligned and corrected here.
- Identity placeholders may be deterministically replaced.
- Timestamp formatting and array normalization may occur.
- Non-fabrication boundaries are enforced.

The corrected surface prepares artifacts for canonical promotion.

3.4C Canonical Promotion and Hash/Freeze Step

The canonical artifacts are generated from the corrected surface.

During canonical promotion:

- `session_baseline.canonical.json` and `session_variant.canonical.json` are produced.
- `module_03_baseline.canonical.json` and `module_03_variant.canonical.json` are produced.
- Structural ordering is normalized.
- Inspection metadata fields are finalized.
- Freeze status indicators are applied.

Only after canonical promotion:

- `hashes.txt` is produced.

- Hash ordering is deterministic.
- Freeze state is declared.

The hash/freeze step is the final act in the inspection cycle.

It marks:

- completion of structural discipline,
- stabilization of inspection surface,
- readiness for archival verification.

Freeze does not modify reasoning content.
It stabilizes structural condition.

3.5 State Diagram (Illustrative Model)

The following diagram illustrates the relationship between operational, inspection, and packaging states:

```
ID_2 Operational States
-----------------------
draft  →  active  →  archived
              |
              | (selection for inspection)
              v

ID_3 Inspection States
----------------------
corrected  →  frozen_under_ID_3
                        |
                        v

Canonical Packaging State
-------------------------
final  (packaging semantics only)
```

Key observations:

- Transition from ID_2 to ID_3 is explicit and intentional.
- Inspection does not alter operational reasoning substance.
- Freeze precedes canonical packaging.
- Packaging finality is structural, not evaluative.

Freeze applies to the canonical surface derived from the corrected layer, while preserving intake provenance in 10_working from ID_2 mode session.

3.6 State Stability and Reuse Discipline

The state model is non-progressive and non-regressive.

It does not represent:

- increasing trust,
- increasing maturity,
- increasing authority.

It represents increasing structural discipline.

Once an artifact pack enters `frozen_under_ID_3` state:

- it is immutable as issued,
- it may not revert to corrected in-place,
- it may not silently re-enter operational posture.

If revision is required:

- a new ID_3 inspection cycle must be initiated, or
- the frozen pack may be copied into a new ID_2 session with a new session identifier.

In such cases:

- the original frozen pack remains unchanged,
- the new operational session becomes a distinct artifact lineage.

All transitions must be explicit and logged.

Silent regression or silent reuse introduces governance risk by collapsing provenance boundaries.

3.7 Relationship to Version E and Version E.2

This state model is consistent with Access-Mode Method - Version E (reasoning posture discipline) and Version E.2 (artifact lifecycle semantics). It operationalizes inspection state transition without altering either framework.

---

4. ID_3 Rule Engine Overview

4.1 Purpose of the Rule Engine

The ID_3 Rule Engine formalizes structural inspection discipline for artifacts selected for freeze.

It exists to:

- Detect structural incompleteness,
- Enforce metadata alignment boundaries,
- Prevent silent structural drift,
- Support deterministic canonical promotion,
- Preserve non-fabrication discipline.

The rule engine does not evaluate governance quality.

It evaluates structural readiness for inspection packaging.

Its function is diagnostic and corrective within defined limits.

4.2 Implementation Form

The Rule Engine is implemented as a structured JSON specification published as a standalone release.

The current inspection rule file is:

`session_completeness.rule.json`

It declares:

- `rule_id`: **CSC_SESSION_COMPLETENESS_V1_1**
- `schema_version`: **rule_v1_1**
- Intended use:
  Diagnostic completeness scoring only. Must not drive content edits beyond minimal metadata correction prior to hash/freeze.

The rule specification defines:

- Required structural constraints
- Optional quality signals

- Validator types
- Weighted completeness scoring
- Non-fabrication boundaries

The Companion introduces the Rule Engine conceptually.
The standalone Rule Engine Formalization release governs:

- Schema evolution
- Version control
- Contribution discipline
- Change log traceability
- Future extension rules

Where discrepancies arise between narrative description and rule specification, the published rule file governs structural enforcement behavior.

4.3 Structural Constraint Categories

The Rule Engine distinguishes between three categories of constraints.

A. Required Structural Constraints

These must exist and validate successfully before freeze.

Examples include:

- Presence of schema_version
- Valid session.session_id
- ISO 8601 UTC timestamps
- Required project context arrays
- Mandatory validation flags

Required constraints block canonical promotion if unmet.

They enforce structural integrity only.

They do not evaluate reasoning substance.

B. Optional Quality Signals

Optional fields contribute to completeness scoring but do not block freeze by default.

Examples include:

- Facilitator details
- Participant role annotations
- Session notes
- Supplementary artifacts

Optional signals improve structural richness and traceability.

They are advisory, not authoritative.

Scoring outputs (e.g., percentage completeness) are diagnostic only.

Completeness scoring must never drive content invention.

C. Non-Fabrication Constraints

Non-fabrication boundaries are the most critical element of the Rule Engine.

The rule specification explicitly prohibits:

- Inventing missing reasoning content,
- Deriving sign-off timestamps from filesystem metadata,
- Reconstructing absent decision records,
- Filling structural gaps with synthetic data.

Permitted corrections include:

- Deterministic replacement of placeholder session identifiers,
- Timestamp normalization formatting,
- Array coercion (scalar → array),
- Structural ordering normalization.

The rule engine may correct structure.
It must not create meaning.

4.4 Diagnostic vs Evaluative Behavior

The Rule Engine is diagnostic.

It:

- Flags missing fields,
- Reports structural misalignment,
- Provides completeness scoring,
- Identifies placeholder violations.

It does not:

- Declare adequacy,
- Validate compliance,
- Approve governance decisions,
- Resolve ambiguity.

Inspection clarity must never be mistaken for validation.

The Rule Engine operates within ID_3 inspection posture only.

4.5 Relationship to ID_2 Artifacts

The Rule Engine interacts with ID_2 artifacts only at the structural level.

When applied:

- ID_2 reasoning remains unchanged.
- Exploratory content is preserved.
- Operational uncertainty remains intact.

If structural correction reveals deeper content inconsistency:

- The artifact must return to ID_2 posture.
- Operational reasoning must resume.
- Inspection cannot repair conceptual gaps.

This prevents inspection from becoming operational reinterpretation.

4.6 Correction Workflow Alignment

The Rule Engine governs the following inspection sequence:

Verify → Correct → Re-Verify → Promote → Freeze → Hash

At each step:

- Structural changes are logged.
- Boundaries are enforced.
- Promotion occurs only after validation passes.

The engine therefore supports deterministic transition from:

corrected → frozen_under_ID_3 → final (packaging semantics).

It is a guardrail mechanism, not a governance evaluator.

4.7 Versioning and Extensibility

The Rule Engine specification includes explicit version identifiers.

Versioning ensures:

- Deterministic interpretation,
- Change traceability,
- Backward compatibility discipline,
- Contribution governance.

Future extensions may include:

- Additional structural validators,
- Mode-aware rule layering,
- Expanded module coverage.

Any extension must preserve:

- Non-fabrication principle,
- Structural-only correction boundary,
- Diagnostic-only scoring semantics.

The Rule Engine must remain posture-bound and non-normative.

4.8 Why Formalization Matters

Without a formal rule specification:

- Structural correction becomes ad hoc,
- Freeze discipline becomes subjective,
- Packaging integrity becomes inconsistent,
- Inspection credibility weakens.

By separating:

- reasoning posture (Version E),
- artifact lifecycle (Version E.2),
- structural inspection enforcement (ID_3 Rule Engine),

the Access-Mode architecture remains layered, interpretable, and regulator-readable.

The Rule Engine formalization strengthens ID_3 as a packaging discipline.

It does not elevate it into a compliance framework.

---

5. Verification Workflow

Operational Sequence Under Rule Governance

5.1 Purpose of the Verification Workflow

The ID_3 verification workflow defines the controlled sequence by which operational artifacts transition into inspection-ready, frozen packs.

The workflow ensures that:

- structural inconsistencies are detected before promotion,
- metadata normalization occurs within defined boundaries,
- canonical artifacts are generated deterministically,
- freeze state is explicitly declared,
- integrity anchoring is applied only after structural discipline is complete.

The workflow is governed by the ID_3 Rule Engine.

It is procedural, but not discretionary.

5.2 Workflow Overview

The ID_3 inspection sequence follows a fixed order:

Verify → Correct → Re-Verify → Canonical Promote → Freeze → Hash

Each step is posture-bound and structurally constrained.

No step may be skipped.

5.3 Step 1 — Verify (Intake Surface)

Verification begins at the `10_working` surface.

This surface:

- contains intake copies of ID_2 `session.json` and module artifacts,
- preserves original metadata,
- remains immutable throughout inspection.

The Rule Engine performs structural diagnostics:

- required field validation,
- array type validation,
- timestamp format validation,
- placeholder detection,
- completeness scoring (diagnostic only).

No modification occurs at this stage.

The objective is detection, not repair.

5.4 Step 2 — Correct (Normalization Surface)

Structural corrections are applied only within the 20_corrected surface.

Permitted corrections include:

- deterministic session identifier replacement (if placeholder),
- timestamp format normalization,
- array coercion,
- structural ordering normalization.

Not permitted:

- fabrication of reasoning content,
- derivation of absent sign-off metadata,
- suppression of recorded gaps,
- inference-based repair.

All corrections remain metadata-bound.

Operational reasoning content is unchanged.

5.5 Step 3 — Re-Verify

After correction, the Rule Engine re-validates the corrected surface.

Promotion may proceed only if:

- required constraints pass,
- structural integrity is confirmed,
- no non-fabrication boundary has been violated.

If re-verification fails:

- the artifact must return to correction,
- or revert to ID_2 posture if conceptual inconsistency is detected.

Inspection does not override reasoning.

5.6 Step 4 — Canonical Promotion

Canonical artifacts are generated from the corrected surface.

The following files are produced:

- `session_baseline.canonical.json`
- `session_variant.canonical.json`
- `module_03_baseline.canonical.json`
- `module_03_variant.canonical.json`

During canonical promotion:

- ordering is normalized,
- inspection metadata is finalized,
- freeze eligibility is confirmed prior to Freeze Declaration.

Canonical artifacts represent the inspection surface only.

They are not operational artifacts.

5.7 Step 5 — Freeze Declaration

Freeze is an explicit act.

At freeze:

- inspection status is set to `frozen_under_ID_3`,
- modification is suspended,
- pack state becomes stable for archival.

Freeze applies to:

- canonical artifacts,
- corrected surface,
- intake surface (for provenance preservation).

Freeze does not imply:

- adequacy,
- compliance,
- approval,
- readiness.

It signals structural stabilization only.

5.8 Step 6 — Hash Generation (Integrity Anchor)

After freeze declaration, the integrity anchor is generated.

`hashes.txt`:

- hashes the canonical artifacts,
- preserves deterministic ordering,
- anchors inspection surface integrity.

The hash applies only to canonical artifacts.

Exploratory ID_2 drafts are not hashed.

Integrity anchoring marks completion of the inspection cycle.

5.9 Outbox Preservation and Provenance

Once frozen and hashed:

- the pack is promoted to the ID_3 outbox,
- `10_working` and `20_corrected` are preserved,
- canonical artifacts and integrity anchor are retained as issued.

The outbox pack:

- is immutable,
- is version-identifiable,
- may serve as inspection reference,
- may serve as baseline input for a new ID_2 session (with explicit new session identity).

Provenance surfaces remain intact.

5.10 Workflow Discipline Summary

The verification workflow enforces:

- structural correction without conceptual alteration,
- deterministic promotion without evaluative judgment,
- freeze without authority inflation,
- integrity anchoring without compliance implication.

The workflow ensures that inspection packaging is:

- reproducible,
- auditable,
- posture-bound,
- and regulator-readable.

It does not transform reasoning into approval.

---

6. Canonical Promotion Semantics

6.1 Purpose of Canonical Promotion

Canonical promotion formalizes the transition from structurally corrected artifacts to inspection-ready artifacts.

It ensures that:

- All structural corrections are finalized,
- Inspection metadata is applied,
- Deterministic ordering is enforced,
- Freeze state is declared,
- Integrity anchoring is performed.

Canonical promotion operates only on artifacts that have successfully passed verification and re-verification under the Rule Engine.

6.2 Inspection Support Artifacts (Pre-Freeze Records)

Before canonical promotion, ID_3 may generate inspection support records.

These may include:

- Metadata correction notes,
- Completeness scoring reports,
- Structural difference summaries,
- Optional-field validation reports,
- Required-field validation confirmations.

For example, a correction log may record:

- Placeholder replacement,
- Timestamp normalization,
- Deterministic session identity alignment.

A completeness report may record:

- Points awarded vs possible,
- Diagnostic percentage score,
- Optional field presence,
- Required field validation status.

These records:

- Support inspection traceability,
- Document structural corrections,
- Provide audit transparency.

They are **not canonical artifacts**.

They are inspection workflow artifacts.

They must not:

- Alter reasoning substance,
- Drive content invention,
- Be treated as adequacy indicators.

Completeness scoring remains diagnostic only.

6.3 Canonical Artifact Generation

Canonical artifacts are derived exclusively from the `20_corrected` surface.

The canonical freeze surface includes:

- `session_baseline.canonical.json`
- `session_variant.canonical.json`
- `module_03_baseline.canonical.json`
- `module_03_variant.canonical.json`

Canonical artifacts:

- Preserve operational reasoning content,
- Reflect metadata normalization,
- Include inspection status fields,
- Are structurally ordered deterministically.

They are distinct from:

- Intake copies in `10_working`,
- Intermediate corrected files,
- Inspection support logs.

6.4 Inspection Status Fields

At canonical promotion, inspection metadata fields are applied:

- `inspection_status = "frozen_under_ID_3"`
- `status = "final"`
- `frozen_at_utc`

These fields indicate structural stabilization.

They do not alter operational reasoning content.

6.5 Freeze Declaration

Freeze is the explicit transition from corrected artifacts to inspection-ready artifacts.

At freeze:

- Canonical artifacts are sealed,
- Inspection status metadata is applied,
- Modification requires version increment.

Freeze stabilizes structural condition.

It does not modify reasoning content or alter operational substance.

6.6 Integrity Anchoring

After freeze declaration, `hashes.txt` is generated.

The integrity anchor:

- Hashes canonical artifacts only,
- Preserves deterministic ordering,
- Enables independent verification,
- Marks completion of the inspection cycle.

Inspection support artifacts (e.g., correction notes, scoring reports) are not part of the canonical hash surface unless explicitly declared in the manifest.

Integrity anchoring establishes:

- Structural reproducibility,
- Tamper evidence,
- Inspection boundary.

It does not establish correctness or compliance.

6.7 Separation of Workflow Records and Canonical Surface

ID_3 produces two distinct artifact classes:

1. **Workflow Records**
    - Correction notes
    - Completeness reports
    - Validation summaries
2. **Canonical Freeze Surface**
    - Canonical JSON artifacts
    - hashes.txt integrity anchor

This separation ensures:

- Inspection transparency,
- Deterministic freeze surface,
- Clear provenance boundary.

Workflow records explain how freeze was achieved.

Canonical artifacts represent what was frozen.

---

7. Deterministic Integrity and Archive Model

7.1 Purpose of the Integrity Model

The Deterministic Integrity and Archive Model defines how frozen ID_3 packs are:

- stabilized,
- verified,
- preserved,
- and reproducible.

Its purpose is structural assurance, not governance validation.

The model ensures that:

- canonical artifacts can be independently verified,
- freeze state cannot be silently altered,
- archival reproduction yields identical hash results,
- provenance surfaces remain intact.

Integrity discipline strengthens inspection credibility without introducing evaluative authority.

7.2 Deterministic Hash Surface

The integrity anchor (`hashes.txt`) is generated only after canonical promotion and freeze declaration.

The hash surface includes:

- `session_baseline.canonical.json`
- `session_variant.canonical.json`
- `module_03_baseline.canonical.json`
- `module_03_variant.canonical.json`

The hash surface excludes:

- exploratory ID_2 drafts,
- workflow correction notes,
- diagnostic scoring reports,
- intermediate corrected files.

This separation ensures that:

- integrity applies to the inspection surface only,
- workflow documentation does not alter freeze determinism,
- operational reasoning remains distinct from inspection packaging.

Deterministic ordering ensures identical canonical artifacts produce identical hashes.

7.3 Verification Procedure

Verification of a frozen pack requires:

1. Recomputing SHA-256 hashes of canonical artifacts
2. Comparing results against hashes.txt
3. Confirming inspection metadata remains unchanged

Verification confirms structural immutability of the canonical surface only.

7.4 Outbox Archive Discipline

After freeze and hash generation, the pack is promoted to the ID_3 outbox.

The outbox pack:

- Is immutable.
- Retains 10_working and 20_corrected for provenance.
- Retains canonical artifacts and integrity anchor.
- Is version-identifiable.
- Is stable as issued.

The outbox serves as:

- Inspection reference snapshot,
- Archival verification unit,
- Baseline candidate for future operational reuse.

Silent modification of an outbox pack invalidates integrity discipline.

If correction is required:

- A new inspection cycle must be initiated.
- A new pack must be generated.

- A new freeze and hash event must occur.

Integrity discipline requires explicit version increments.

7.5 Reproducibility Expectations

Reproducibility under ID_3 requires:

- Deterministic JSON ordering,
- Stable metadata application,
- No runtime-dependent fields,
- No environment-derived variation,
- Explicit freeze timestamps,
- Controlled identity derivation.

A re-run of canonical promotion using identical corrected artifacts must produce:

- Identical canonical JSON,
- Identical hash values.

If reproduction yields divergence:

- Structural ordering must be examined,
- Timestamp discipline must be reviewed,
- Non-deterministic generation must be eliminated.

Reproducibility strengthens inspection defensibility.

It does not establish governance validity.

7.6 Integrity vs Authority

Hash stability confirms structural immutability and reproducibility.

It does not confirm governance validity, policy effectiveness, or compliance standing.

Integrity discipline protects inspection interpretability.

Authority remains external to the integrity model.

7.7 Interaction with Future Sessions

A frozen and archived pack may serve as input to a new ID_2 operational session under strict conditions:

- The original frozen pack remains unchanged.
- A new session identifier is generated.
- Operational reasoning resumes under ID_2 posture.
- Any new inspection cycle generates a new pack and integrity anchor.

This preserves:

- Lineage clarity,
- Provenance continuity,
- State boundary discipline.

Integrity must remain explicit across sessions.

7.8 Governance Value of Deterministic Archiving

The integrity and archive model provides:

- Tamper-evident inspection surfaces,
- Clear provenance boundaries,
- Deterministic reproduction,
- Regulator-readable stabilization signals.

It transforms inspection packaging into a controlled governance control layer.

It does not elevate ID_3 into an assurance framework.

---

8. Provenance and Reproducibility

8.1 Purpose of Provenance Discipline

Provenance discipline ensures that:

- The origin of artifacts remains traceable,
- Structural corrections are inspectable,
- Freeze state is auditable,
- Artifact lineage is preserved across sessions.

Provenance is not a quality metric.
It is a transparency mechanism.

ID_3 preserves provenance explicitly through layered surfaces rather than through silent transformation.

8.2 10_working as Evidentiary Intake Surface

The 10_working surface serves as the evidentiary intake layer.

It contains:

- Direct copies of ID_2 session.json artifacts,
- Direct copies of module artifacts (e.g., module_03),
- Original metadata as received from ID_2 posture.

Characteristics:

- No normalization occurs here.
- No structural edits are applied.
- Files remain immutable once inspection begins.
- It preserves the pre-inspection condition of the artifacts.

The purpose of 10_working is evidentiary stability.

It allows a reviewer to compare:

- Intake condition
- Corrected condition
- Canonical freeze condition

Without reconstructing intent.

8.3 20_corrected as Normalization Boundary

The 20_corrected surface exists as a controlled structural boundary.

It contains:

- Metadata-normalized versions of session artifacts,
- Deterministic identity replacement (where placeholder values exist),
- Timestamp normalization,
- Array coercion and structural alignment.

Characteristics:

- Correction is metadata-bound only.
- Non-fabrication constraints apply.
- Operational reasoning content remains unchanged.
- Structural adjustments are inspectable and reproducible.

The corrected surface prepares artifacts for canonical promotion.

It does not represent freeze state.

It does not represent evaluation.

It represents structural alignment only.

8.4 Canonical Surface as Freeze Record

The canonical surface represents the inspection-ready freeze record.

It contains:

- Canonical JSON artifacts,
- Explicit inspection metadata,
- Deterministic structural ordering,
- Integrity anchor (hashes.txt).

The canonical surface:

- Is stable as issued,
- Is reproducible,
- Is hash-verifiable,
- Is version-identifiable.

It is the formal inspection reference.

It is distinct from both intake and correction layers.

8.5 Lineage Across Sessions

Each inspection cycle produces an immutable pack.

New operational sessions must:

- Generate new session identifiers
- Initiate new inspection cycles
- Produce new integrity anchors

Lineage is explicit and non-regressive.

8.6 Reuse as New ID_2 Baseline

A frozen ID_3 pack may serve as input to a new ID_2 session under controlled conditions:

- The original pack remains unchanged.
- Intake is performed into a new operational session.
- A new session identity is generated.
- New inspection cycles produce new freeze artifacts.

Reusing canonical artifacts does not:

- Transfer inspection status,
- Transfer adequacy,
- Transfer compliance implications.

Each inspection cycle stands independently.

8.7 Provenance vs Authority

Provenance discipline ensures:

- Structural transparency,
- Lineage clarity,
- Explicit state transition,
- Traceable inspection boundaries.

Provenance does not establish correctness or adequacy.

It preserves interpretability across artifact generations.

---

9. Troubleshooting and Structural Edge Cases

9.1 Purpose of Troubleshooting Discipline

Troubleshooting under ID_3 is structural, not conceptual.

Its purpose is to:

- Identify schema inconsistencies,
- Detect metadata misalignment,
- Prevent silent structural corruption,
- Enforce non-fabrication boundaries.

Troubleshooting does not:

- Repair reasoning substance,
- Resolve governance ambiguity,
- Supply missing operational content,
- Validate adequacy.

If structural correction reveals conceptual inconsistency, the artifact must return to ID_2 posture.

9.2 Array Scalarization Issues

Problem

Operational sessions may serialize array fields as scalars.

Examples:

- `participants` serialized as a single object rather than an array,
- `validation_events` serialized as a single entry,
- `secondary_modes_allowed` represented as a string.

This creates schema misalignment and validator failure.

Permitted Correction

Under ID_3:

- Scalars may be coerced into arrays,

- Array shape may be normalized,
- Ordering may be standardized.

The correction must:

- Preserve original values,
- Avoid insertion of synthetic entries,
- Remain metadata-bound.

Array coercion is structural correction, not content modification.

9.3 Timestamp Coercion Issues

Problem

Timestamps may:

- Be stored as local time,
- Lack timezone indicators,
- Appear as DateTime objects,
- Be inconsistently formatted.

Permitted Correction

Under ID_3:

- Timestamps may be normalized to ISO 8601 UTC format,
- Explicit timezone indicators may be applied,
- Structural formatting may be corrected.

Not permitted:

- Inventing missing timestamps,
- Deriving sign-off timestamps from filesystem metadata,
- Inferring decision timing not explicitly recorded.

Timestamp normalization ensures structural consistency.

It must not create historical claims.

9.4 Placeholder Identity Handling

Problem

ID_2 drafts may retain placeholder identifiers such as:

```
ID2-YYYYMMDD-HHMM-<shortname>
```

Placeholder identifiers undermine freeze integrity.

Permitted Correction

Under ID_3:

- Placeholder identifiers may be deterministically replaced,
- Replacement must derive from folder-derived or declared session identity,
- Replacement logic must be reproducible.

Not permitted:

- Inventing session identities without traceable source,
- Overwriting valid non-placeholder identifiers,
- Deriving identities from implicit assumptions.

Identity correction must be deterministic and logged.

9.5 Optional Field Emptiness

Optional fields may be:

- Present but empty,
- Present but invalid,
- Omitted entirely.

Optional emptiness:

- Does not block freeze,
- May affect diagnostic completeness scoring,
- Must not trigger synthetic population.

Inspection may record emptiness.

It must not conceal or compensate for it.

9.6 Schema Drift Detection

Schema drift occurs when:

- Field names differ across baseline and variant,
- Structural nesting changes,
- Required keys are renamed or omitted.

Drift may result from:

- Manual editing,
- Tool variation,
- Prior experimental sessions.

When schema drift is detected:

- Required fields must be restored,
- Structural alignment must be re-established,
- Re-verification must occur before promotion.

If drift reflects conceptual change rather than structural error:

- The artifact must revert to ID_2 posture,
- Operational reasoning must resume,
- A new inspection cycle must be initiated.

9.7 Completeness Scoring Misinterpretation

Completeness scoring is diagnostic only.

High completeness:

- Does not imply adequacy,
- Does not imply correctness,
- Does not imply inspection readiness without structural validation.

Low completeness:

- Does not invalidate reasoning,
- May indicate optional metadata absence,

- May indicate legitimate operational context.

Completeness metrics must not drive content invention.

9.8 When to Revert to ID_2

ID_3 must halt and revert to ID_2 posture when:

- Structural correction reveals conceptual inconsistency,
- Required content is substantively missing,
- Ambiguity cannot be resolved without reasoning modification,
- Freeze would conceal operational uncertainty.

Inspection must not:

- Repair governance logic,
- Rewrite scenario analysis,
- Resolve unresolved decisions.

If structural correction would alter reasoning substance, the artifact MUST revert to ID_2 posture.

9.9 Freeze Invalidation Scenarios

Freeze discipline may be invalidated if:

- Canonical artifacts are modified after hash generation,
- Outbox pack is altered without version increment,
- Hash ordering becomes non-deterministic,
- Freeze timestamps are overwritten,
- Inspection metadata is edited post-freeze.

In such cases:

- The pack must be treated as compromised,
- A new inspection cycle must be performed,
- A new freeze and hash event must be generated.

Integrity discipline requires explicit correction.

Silent modification collapses provenance.

9.10 Structural Troubleshooting Summary

ID_3 troubleshooting discipline ensures:

- Structural correction without conceptual interference,
- Deterministic freeze without fabrication,
- Explicit reversion when reasoning changes are required.

Structural alignment is necessary for inspection.

Conceptual alignment remains an ID_2 responsibility.

ID_3 inspection cannot repair reasoning generated under ID_2 posture.

10. Operational Discipline Statement

10.1 ID_3 as Inspection Packaging Discipline

Access-Mode ID_3 is an inspection packaging discipline.

It governs:

- Structural alignment,
- Metadata normalization,
- Canonical promotion,
- Freeze declaration,
- Integrity anchoring,
- Archive stabilization.

It does not govern:

- Governance quality,
- Operational adequacy,
- Risk sufficiency,
- Legal interpretation,
- Compliance determination,
- Certification or approval.

ID_3 formalizes how artifacts are prepared for inspection.
It does not elevate the authority of those artifacts.

10.2 Explicit Non-Claims

The following claims must not be inferred from ID_3 usage:

- Regulatory compliance
- Standards conformance
- Governance adequacy
- Organizational readiness
- Policy effectiveness
- Control sufficiency
- Ethical validation

Freeze state and hash stability indicate structural reproducibility only.

They do not signal trust, correctness, or approval.

Completeness scoring does not imply maturity.

Inspection clarity does not imply validation.

Determinism does not imply sufficiency.

ID_3 remains non-normative.

10.3 Compatibility with Standards Preparation

ID_3 supports standards-facing work by improving:

- Structural clarity,
- Traceability,
- Reproducibility,
- Posture discipline,
- Inspection readability.

It is compatible with:

- ISO-style documentation discipline,
- Regulator-facing review contexts,
- Audit preparation workflows,
- Standards exploration exercises.

However, ID_3:

- Does not implement standards,
- Does not evaluate conformance,
- Does not substitute for assurance frameworks,
- Does not create certification pathways.

It operates alongside governance frameworks — not in place of them.

10.4 Human Authority and Review Requirement

All outputs generated under ID_3 remain draft artifacts.

Human authority is final and non-delegable.

Humans remain responsible for:

- Session selection,
- Mode declaration,
- Structural correction decisions,
- Freeze declaration,
- Release publication,
- External interpretation,
- Governance conclusions.

AI systems may assist with:

- Structural inspection,
- Metadata normalization,
- Artifact organization.

They may not:

- Grant approval,
- Resolve normative disputes,
- Declare adequacy,
- Substitute for human judgment.

Inspection discipline reinforces human control.
It does not automate governance authority.

10.5 Final Positioning

ID_3 transforms working governance artifacts into inspection-ready artifacts through deterministic structural discipline.

It ensures that:

- Reasoning posture remains distinct from packaging semantics,
- Artifact state remains distinct from authority,
- Freeze remains distinct from approval,
- Integrity remains distinct from validation.

By separating:

- Operational reasoning (ID_2),
- Inspection packaging (ID_3),

- Artifact lifecycle semantics (Version E.2),
- Reasoning posture control (Version E),

the Access-Mode architecture remains layered, interpretable, and regulator-readable.

ID_3 strengthens clarity.

It does not elevate trust.

Governance authority remains human.

Executive Closing

Access-Mode ID_3 formalizes inspection packaging as a disciplined, deterministic governance control. By separating operational reasoning from structural freeze and integrity anchoring, it preserves interpretability without inflating authority. ID_3 strengthens artifact clarity and reproducibility while keeping governance judgment explicitly human and non-delegable.
